

Strings

September 30, 2025

1 Introducción a la Programación para Ciencia de Datos

1.1 Lenguaje de programación R

Rocío Romero Zaliz - rocio@decsai.ugr.es

2 Cadenas de caracteres en R

Aunque R es un lenguaje estadístico en el que los vectores numéricos y las matrices desempeñan un papel central, las cadenas de caracteres también son necesarias y R dispone de una serie de utilidades de manipulación de cadenas.

```
[ ]: texto <- "Ciencia de datos"  
      class(texto)
```

```
[ ]: length(texto)
```

```
[ ]: nchar(c("Ciencia de Datos", "a"))
```

```
[ ]: # String construction  
  
      empty_str <- character(10)  
      print(empty_str)
```

```
[ ]: print(paste("Ciencia", "de", "Datos"))
```

```
[ ]: paste("Ciencia", "de", "Datos", sep="_|_")
```

```
[ ]: paste("Ciencia", c("hola", "mundo"), "cierta")
```

```
[ ]: paste(1:3, 1:5, sep="_")
```

```
[ ]: paste(1:3, 1:5, sep="_", collapse="|")
```

```
[ ]: ?paste
```

2.1 Ejemplo

- Dado un conjunto de datos, anonimizar los nombres de las columnas

```
[ ]: data <- data.frame(name = c("Susan", "Greg", "Amy", "Laura", "David"),
                        lastname = c("Wilson", "Gray", "Sanders", "Xeon", "Rogers"),
                        gender = factor(c("F", "M", "F", "F", "M")), age = c(23, 46, 32, 90, 53))
data
```

```
[ ]: colnames(data)
```

```
[ ]: colnames(data) <- paste("var", 1:dim(data)[2], sep="_")
data
```

2.2 Funciones básicas para trabajar con cadenas de caracteres

```
[ ]: substr("Ciencia de Datos", 3, 5)
```

```
[ ]: substr("Ciencia de Datos", 12) # Error!
```

```
[ ]: date()
```

```
[ ]: c(date(), "hola mundo")
```

```
[ ]: print(strsplit(c(date(), "hola mundo"), split=" ")
```

```
[ ]: class(strsplit(c(date(), "hola mundo"), split=" ")
```

```
[ ]: print(strsplit(c("Esta es una frase", "Esta es otra linda frase"), split="a ")
```

```
[ ]: print(strsplit(c("Esta es una frase", "Esta es otra frase"), split=c(" ", "e")))
```

2.3 Ejemplo

```
[ ]: files <- list.files()
print(files)
```

```
[ ]: print(strsplit(files, split="."))
```

```
[ ]: print(strsplit(files, split="\\."))
```

```
[ ]: print(unlist(strsplit(files, split="\\.")))
```

```
[ ]: matrix(unlist(strsplit(files, split="\\.")), ncol=2, byrow=TRUE)
```

2.4 Expresiones regulares

- Una expresión regular es una especie de comodín.
- Es una forma abreviada de especificar distintas clases de cadenas de caracteres.
- En R, debes prestar atención a este punto cuando uses las funciones `grep`, `grepl`, `regexr`, `gregexpr`, `sub`, `gsub` y `strsplit`.

```
[ ]: date()
```

```
[ ]: print(strsplit(date(), split="[0-9]+"))
```

```
[ ]: print(files)
```

```
[ ]: grep("pdf", files)
```

```
[ ]: files[grep("pdf", files)]
```

```
[ ]: donde <- grep("[ia]", c("Ciencia","de","Datos"))
donde
```

```
[ ]: c("Ciencia","de","Datos")[donde]
```

```
[ ]: grep("e.", c("Ciencia","de","Datos"))
```

```
[ ]: print(strsplit("a.b.c", "."))
```

```
[ ]: print(strsplit("a.b.c", "\\."))
```

```
[ ]: print(strsplit("a.b.c", "[.]")) # ¡Cuidado!
```

2.5 Ejemplo

Anonimización de datos

```
[ ]: data
```

```
[ ]: ncolumn <- paste(tolower(substr(data$var_1, 1, 1)),
                     tolower(substr(data$var_2, 1, 1)), sep = "")
ncolumn
```

```
[ ]: paste(colnames(data)[1:2], collapse="_and_")
```

```
[ ]: data$var_1 <- ncolumn
colnames(data)[1] <- paste(colnames(data)[1:2], collapse="_and_")
data <- data[,-2]
data
```

More info in: [R Manual](#)

2.6 Ejercicios strings (1ra parte)

1. Crea un vector de cadenas de caracteres con tu nombre y apellidos (por ejemplo, ["Rocío", "Romero", "Zaliz"]). A partir de el crea una nueva cadena de caracteres con la inicial de tu nombre, un punto y tus apellidos (por ejemplo, "R. Romero Zaliz").
2. Dado un vector de cadenas de caracteres que representan fechas (por ejemplo, ["2005-11-28", "2015-10-18", "2000-01-01"], utilizando el formato AÑO-MES-DÍA), mostrar sólo las correspondientes a los meses impares.
3. Dada una cadena de caracteres con varias palabras (por ejemplo, "Esta es una frase, pero no cualquier frase.") crea un vector con cada una de las palabras de la cadena (por ejemplo, ["Esta", "es", "una", "frase", "pero", "no", "cualquier", "frase"]). Tenga en cuenta todos los caracteres de puntuación posibles.
4. Busca en un vector de cadenas de caracteres aquellas que incluyan sólo vocales "a" y/o "e" o ninguna (comprueba mayúsculas y minúsculas, considera á, é, Á y É como otros caracteres y no los que buscas).
5. Dados tres vectores numéricos que representan días, meses y años, crea un vector nuevo con fechas (sólo si son válidas, si la fecha es inválida ignorarla) (Sugerencia: investigue la función `as.Date`).

3 Manipulación de cadenas de caracteres usando Tidyverse

3.1 Cheat sheets

- <https://github.com/rstudio/cheatsheets/raw/main/strings.pdf>

```
[ ]: library(tidyverse)
#library(stringr)

data <- data.frame(name = c("Susan", "Greg", "Amy", "Laura", "David"), lastname_
  ↪ c("Wilson", "Gray", "Sanders", "Xeon", "Rogers"), gender = factor(c("F",
  ↪ "M", "F", "F", "M")), age = c(23, 46, 32, 90, 53))
data
```

La función `nchar` `str_length` indica la longitud de la cadena de caracteres.

```
[ ]: data %>% pull(lastname) %>% str_length
```

La función `paste` `str_c` concatena varias cadenas de caracteres.

```
[ ]: print(data$name %>% str_c(collapse = ", "))
[ ]: print(str_c(data$name, data$lastname, sep = " - "))
[ ]: ?unite
[ ]: data %>% unite("Full_name", c(name, lastname), sep = " - ")
[ ]: data %>% unite("Full_name", c(name, lastname), remove = FALSE, sep = " - ")
[ ]: data %>% unite("Full_name", c(name, lastname), remove = FALSE, sep = " - ") %>%
      separate(Full_name, c("x", "y"), sep = " - ")
```

La función `substr(x, inicio, parada)` `str_sub(x, inicio, parada)` devuelve la subcadena en el rango de posiciones de caracteres dado inicio:parada para la cadena x.

```
[ ]: data$name %>% str_sub(2, 3)
```

3.2 Expresiones regulares

La función `str_detect` indica si hay alguna coincidencia con el patrón indicado.

```
[ ]: data[1,]
[ ]: data[1,] %>% str_detect("[ae]+")
[ ]: data$name
[ ]: data$name %>% str_detect("[aeiou]")
```

La función `str_subset` extrae los componentes que han coincidido.components

```
[ ]: data$name %>% str_subset("[aeiou]")
```

La función `str_count` cuenta la cantidad de veces que coincide.

```
[ ]: data$name %>% str_count("[aeiou]")
```

La función `str_extract` extrae el texto de la coincidencia.

```
[ ]: print(data$name %>% str_extract("[aeiou]"))
[ ]: print(data$name %>% str_extract_all("[aeiou]"))
```

La función `str_match` extrae partes de la coincidencia definida entre paréntesis.

```
[ ]: print(data$name)

data$name %>% str_match("(.)[aeiou](.)")
```

```
[ ]: print(data$name %>% tolower %>% str_match_all("[aeiou](.)"))
```

La función `str_replace` reemplaza las coincidencias por una nueva cadena de caracteres.

```
[ ]: data$name %>% str_replace("[aeiou]", "*")
```

```
[ ]: data$name %>% str_replace_all("[aeiou]", "*")
```

La función `strsplit` divide una cadena de caracteres en trozos.

```
[ ]: print(data$name %>% str_split("[aeiou]"))
```

3.3 Ejemplo

```
[ ]: #install.packages("ISLR")
library("ISLR")
College
```

```
[ ]: college.names <- rownames(College)
college.names
```

1. Obtener un vector que contenga todas las universidades con **Texas** en su nombre. ¿Cuántas hay?

```
[ ]: college.names %>% str_count("Texas") %>% sum
```

2. Obtener un vector de todas las filas del conjunto de datos `College` que contengan el término “University” (HINT: `str_which`).

```
[ ]: college.names %>% str_which("University")
```

3. ¿Cuántas “University” hay en el conjunto de datos frente a “College”?

```
[ ]: college.names %>% str_count("University") %>% sum
```

```
[ ]: college.names %>% str_count("College") %>% sum
```

```
[ ]: 345+406 == length(college.names)
```

```
[ ]: pos_c <- college.names %>% str_which("College")
pos_u <- college.names %>% str_which("University")

college.names[setdiff(1:length(college.names), c(pos_c, pos_u))]
```

```
[ ]: college.names[!str_detect(college.names, "Coll|Univ")]
```

3.4 Ejercicios string (2da parte)

Lo mismo que en la parte 1 pero usando las funciones de **stringr**.

1. Crea un vector de cadenas de caracteres con tu nombre y apellidos (por ejemplo, ["Rocío", "Romero", "Zaliz"]). A partir de el crea una nueva cadena de caracteres con la inicial de tu nombre, un punto y tus apellidos (por ejemplo, "R. Romero Zaliz").
2. Dado un vector de cadenas de caracteres que representan fechas (por ejemplo, ["2005-11-28", "2015-10-18", "2000-01-01"], utilizando el formato AÑO-MES-DÍA), mostrar sólo las correspondientes a los meses impares.
3. Dada una cadena de caracteres con varias palabras (por ejemplo, "Esta es una frase, pero no cualquier frase.") crea un vector con cada una de las palabras de la cadena (por ejemplo, ["Esta", "es", "una", "frase", "pero", "no", "cualquier", "frase"]). Tenga en cuenta todos los caracteres de puntuación posibles.
4. Busca en un vector de cadenas de caracteres aquellas que incluyan sólo vocales "a" y/o "e" o ninguna (comprueba mayúsculas y minúsculas, considera á, é, Á y É como otros caracteres y no los que buscas).
5. Dados tres vectores numéricos que representan días, meses y años, crea un vector nuevo con fechas (sólo si son válidas, si la fecha es inválida ignorarla) (Sugerencia: investigue la función `as.Date`).

4 References

- Gaston Sanchez. Handling and Processing Strings in R. https://www.gastonsanchez.com/Handling_and_Processing_Strings_in_R.pdf
- R-tutorials. <http://r-tutorials.com>
- Norman Matloff. 2011. The Art of R Programming: A Tour of Statistical Software Design (1st ed.). No Starch Press, San Francisco, CA, USA.
- Patrick Burns. 2011. The R Inferno.